

Unit -4

Android Activity

An **Activity** in Android is a single screen in an app with which users can interact. It serves as the primary building block of an Android application, managing the user interface and handling user interactions.

The Activity life cycle

The Activity Lifecycle in Android refers to the sequence of states that an activity goes through during its existence. Understanding the lifecycle is crucial for managing the state of the activity, handling user interactions, and optimizing app performance. The primary states in the Activity Lifecycle include:

1. onCreate():

- **Purpose:** Called when the activity is first created.
- **Usage:** Initialize components like views, data binding, and other initial setup.
- **Example:** `setContentView(R.layout.activity_main);`

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main); // Setting up the layout
    Button myButton = findViewById(R.id.myButton); // Initializing a
    button
    myButton.setOnClickListener(v -> {
        // Handle button click
    });
}
```

2. onStart():

- **Purpose:** Invoked when the activity becomes visible to the user.
- **Usage:** Perform tasks that need to be done each time the activity enters the foreground.
- **Example:** Registering a broadcast receiver.

```
@Override
protected void onStart() {
    super.onStart();
    // Registering a broadcast receiver to listen for specific events
    IntentFilter filter = new
    IntentFilter(Intent.ACTION_BATTERY_CHANGED);
    registerReceiver(batteryReceiver, filter);
}
```

3. onResume():

- **Purpose:** Called when the activity starts interacting with the user.
- **Usage:** Resume UI updates, animations, or any functionality that was paused.
- **Example:** Resuming a paused game.

```

@Override
protected void onResume() {
    super.onResume();
    // Resuming a paused game or animation
    gameView.resumeGame(); // Assuming there's a method to resume the
game
}

```

4. onPause():

- **Purpose:** Triggered when the system is about to resume another activity.
- **Usage:** Save data, stop animations or other CPU-intensive tasks.
- **Example:** Pausing a video playback.

```

@Override
protected void onPause() {
    super.onPause();
    // Pausing video playback or saving data before the activity is
paused
    videoPlayer.pause(); // Assuming there's a video player object
    saveGameData(); // Save game data to prevent loss
}

```

5. onStop():

- **Purpose:** Called when the activity is no longer visible.
- **Usage:** Release resources that are not needed when the activity is not visible.
- **Example:** Unregistering a broadcast receiver.

```

@Override
protected void onStop() {
    super.onStop();
    // Unregistering the broadcast receiver to save resources
    unregisterReceiver(batteryReceiver);
}

```

6. onRestart():

- **Purpose:** Invoked when the activity is being restarted after stopping.
- **Usage:** Used to reinitialize components that were released or reset in onStop().
- **Example:** Re-acquiring resources released in onStop().

```

@Override
protected void onRestart() {
    super.onRestart();
    // Re-acquiring resources or refreshing the UI
    IntentFilter filter = new
IntentFilter(Intent.ACTION_BATTERY_CHANGED);
    registerReceiver(batteryReceiver, filter); // Re-registering the
broadcast receiver
}

```

7. **onDestroy():**

- **Purpose:** Called before the activity is destroyed.
- **Usage:** Perform final cleanup, such as freeing up resources.
- **Example:** Canceling any ongoing tasks, saving persistent data.

```
@Override
protected void onDestroy() {
    super.onDestroy();
    // Canceling any ongoing tasks and freeing resources
    if (downloadTask != null) {
        downloadTask.cancel(true); // Assuming there's a download task
        running
    }
    savePersistentData(); // Save any persistent data before the
    activity is destroyed
}
```

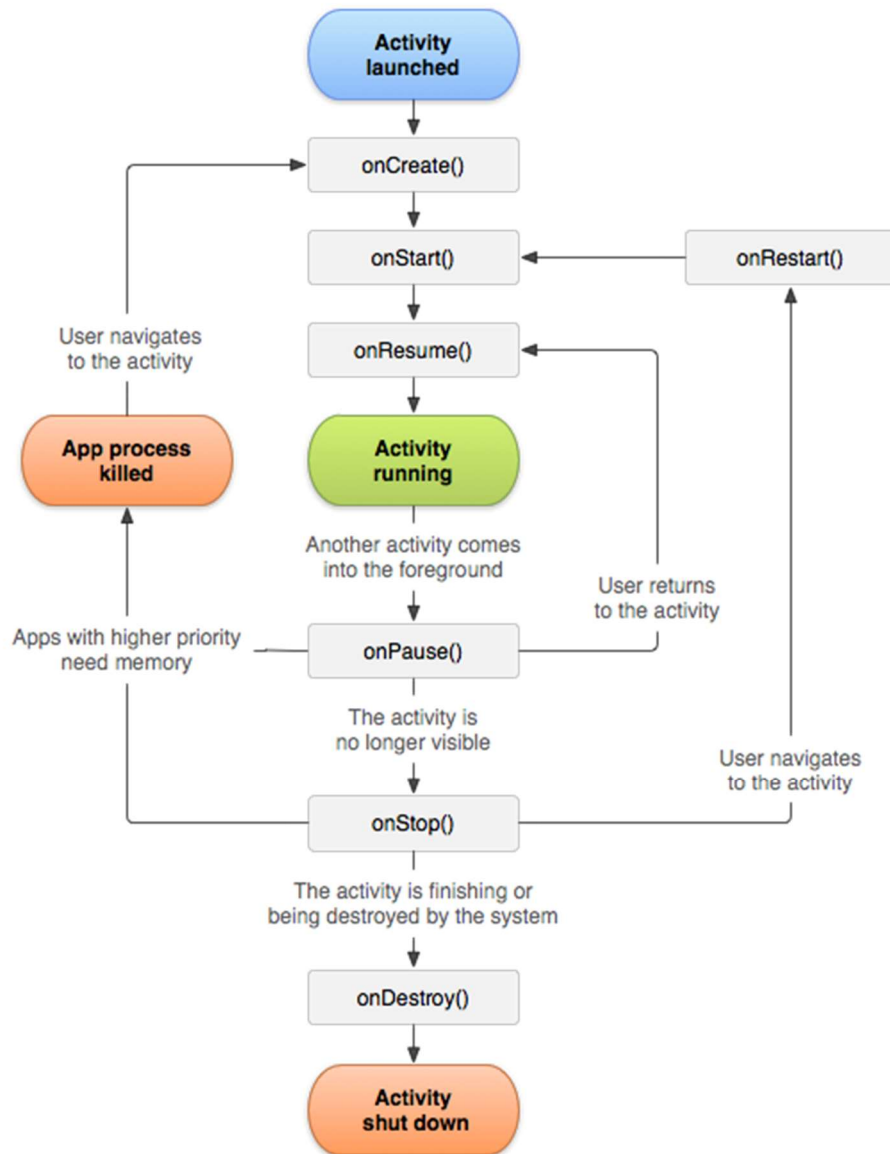


Fig: Android Activity Lifecycle

Here's a simple implementation of an Android Activity that demonstrates the use of lifecycle methods:

```

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d("ActivityLifecycle", "onCreate called");
    }

    @Override

```

```

protected void onStart() {
    super.onStart();
    Log.d("ActivityLifecycle", "onStart called");
}

@Override
protected void onResume() {
    super.onResume();
    Log.d("ActivityLifecycle", "onResume called");
}

@Override
protected void onPause() {
    super.onPause();
    Log.d("ActivityLifecycle", "onPause called");
}

@Override
protected void onStop() {
    super.onStop();
    Log.d("ActivityLifecycle", "onStop called");
}

@Override
protected void onRestart() {
    super.onRestart();
    Log.d("ActivityLifecycle", "onRestart called");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d("ActivityLifecycle", "onDestroy called");
}
}

```

Creating multiple activities

In an Android application, you can create multiple activities to represent different screens. Each activity functions independently and can be launched from another activity or directly from a notification, widget, or other components.

Steps to Create Multiple Activities:

1. Creating a New Activity:

- To create a new activity in Android Studio, right-click on your project directory (app > java > your.package.name), choose New > Activity, and select the desired type of activity (e.g., Empty Activity).
- Name the activity (e.g., SecondActivity) and ensure that the Generate Layout File and Launcher Activity checkboxes are appropriately checked.

- Android Studio will automatically create a new Java/Kotlin class for the activity (e.g., `SecondActivity.java`) and a corresponding XML layout file (e.g., `activity_second.xml`).

2. Updating `AndroidManifest.xml`:

- Each activity in your app must be declared in the `AndroidManifest.xml` file.
- When you create a new activity in Android Studio, it automatically updates the manifest file to include the new activity.
- Example:

```
<activity android:name=".SecondActivity"></activity>
```

3. Navigating Between Activities:

- Use an `Intent` to navigate between activities. An intent is an abstract description of an operation to be performed.
- Example of starting a new activity

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

You can pass data between activities using `Intent` extras:

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra("EXTRA_MESSAGE", "Hello from MainActivity!");
startActivity(intent);
```

4. Receiving Data in the Second Activity:

- In `SecondActivity`, you can retrieve the data passed from the first activity using `getIntent()`:

```
Intent intent = getIntent();
String message = intent.getStringExtra("EXTRA_MESSAGE");
TextView textView = findViewById(R.id.textView);
textView.setText(message);
```

Example: Creating and Using Multiple Activities

MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button = findViewById(R.id.button);
        button.setOnClickListener(v -> {
            // Navigate to SecondActivity
            Intent intent = new Intent(MainActivity.this,
SecondActivity.class);
            intent.putExtra("EXTRA_MESSAGE", "Hello from
MainActivity!");
            startActivity(intent);
        });
    }
}

```

activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go to Second Activity" />
</RelativeLayout>

```

SecondActivity.java

```

public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        // Get the Intent that started this activity and extract the
string
        Intent intent = getIntent();
        String message = intent.getStringExtra("EXTRA_MESSAGE");

        // Capture the layout's TextView and set the string as its text
        TextView textView = findViewById(R.id.textView);
        textView.setText(message);
    }
}

```

activity_second.xml

```

<?xml version="1.0" encoding="utf-8"?>

```

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is the second activity" />
</RelativeLayout>
```

Getting a result back from a child activity

When working with multiple activities, you may want to start a child activity and receive a result back in the parent activity. This is useful when you need to get user input or make a selection in a child activity and use that result in the parent activity.

Steps to Get a Result Back from a Child Activity:

1. Start Child Activity for Result:

- Instead of using `startActivity()`, use `startActivityForResult()` to start the child activity.
- Example

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivityForResult(intent, REQUEST_CODE);
```

- `REQUEST_CODE` is an integer constant that identifies the request.

1. Send Result Back from Child Activity:

- In the child activity, use `setResult()` to send data back to the parent activity before finishing.
- Example:

```
Intent resultIntent = new Intent();
resultIntent.putExtra("resultKey", "This is the result");
setResult(RESULT_OK, resultIntent);
finish(); // Close the child activity
```

2. Handle the Result in Parent Activity:

- Override `onActivityResult()` in the parent activity to handle the result.
- Example:

```
@Override
```



```
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == REQUEST_CODE && resultCode == RESULT_OK) {
        String result = data.getStringExtra("resultKey");
        // Use the result (e.g., update UI)
        TextView resultTextView = findViewById(R.id.resultTextView);
        resultTextView.setText(result);
    }
}
```

Example: Returning a Result from a Child Activity

[MainActivity.java](#)

```
public class MainActivity extends AppCompatActivity {
    private static final int REQUEST_CODE = 1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button button = findViewById(R.id.button);
        button.setOnClickListener(v -> {
            Intent intent = new Intent(MainActivity.this, SecondActivity.class);
            startActivityForResult(intent, REQUEST_CODE);
        });
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode == REQUEST_CODE && resultCode == RESULT_OK) {
            String result = data.getStringExtra("resultKey");
            TextView resultTextView = findViewById(R.id.resultTextView);
            resultTextView.setText(result);
        }
    }
}
```

[SecondActivity.java](#)

```
public class SecondActivity extends AppCompatActivity {
```

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_second);

    Button returnButton = findViewById(R.id.returnButton);
    returnButton.setOnClickListener(v -> {
        Intent resultIntent = new Intent();
        resultIntent.putExtra("resultKey", "Hello from SecondActivity!");
        setResult(RESULT_OK, resultIntent);
        finish();
    });
}
}

```

[activity_main.xml](#)

```

<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Second Activity" />

    <TextView
        android:id="@+id/resultTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/button"
        android:text="Result will be displayed here" />
</RelativeLayout>

```

[activity_second.xml](#)

```

<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/returnButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Return Result" />
</RelativeLayout>

```

Getting and Setting Data to/from the Layout File

In Android, you interact with your layout files using `findViewById()` to get references to UI components and then set or retrieve their data programmatically.

Accessing and Modifying Layout Components:

1. Getting Data from Layout Components:

- Use `findViewById()` to get a reference to a UI element.
- Example:

```
EditText editText = findViewById(R.id.editText);  
String text = editText.getText().toString(); // Getting text from EditText
```

2. Setting Data to Layout Components:

- After obtaining a reference to the UI component, you can set its data.
- Example:

```
TextView textView = findViewById(R.id.textView);  
textView.setText("New Text"); // Setting new text to TextView
```

Example: Setting and Getting Data from Layout Components

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        // Accessing the EditText and getting its text  
        EditText editText = findViewById(R.id.editText);  
        String inputText = editText.getText().toString();  
  
        // Accessing the TextView and setting its text  
        TextView textView = findViewById(R.id.textView);  
        textView.setText(inputText);  
  
        // Example of setting a Button click listener to update the TextView  
        Button button = findViewById(R.id.button);  
        button.setOnClickListener(v -> {  
            String updatedText = editText.getText().toString();
```

```
        textView.setText(updatedText);
    });
}
}
```

activity_main.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/editText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter text here" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Update TextView" />

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This will be updated" />
</LinearLayout>
```